

Department of Information Technology

Delhi Technological University

Vision:

To be a centre of excellence in education and research, producing Creators of Innovative Technology to serve humanity.

Mission:

The Mission of the department is as follows:

M1: To develop human resources with sound knowledge and intellectual capability for innovation, research and excellence in the field of Information Technology.

M2: To provide conducive academic environment for achieving world class standards in teaching and cutting-edge research in the field of Information Technology.

M3: To inculcate professional ethics and social responsibility in students and faculty to contribute in the overall development of society.

M4: To facilitate the development of academia-industry collaborations and societal outreach programmes.

Department of Information Technology

Delhi Technological University

Operating Systems (IT204) Lab

Course Objective:

To familiar with the Fundamental Principles of the OS, its services and Functionalities, the concepts of processes, process synchronization, and scheduling, memory management and need for protection in computer systems.

Course Outcomes:

- CO1:** To describe functions, structures and design issues of operating systems.
- CO2:** To apply various process management concepts including scheduling.
- CO3:** To demonstrate concepts of memory management including virtual memory.
- CO4:** To implement system resources sharing among the users.
- CO5:** To describe issues related to file system interface and implementation, disk management.
- CO6:** To discuss and compare different types of operating systems including Unix

Department of Information Technology

Delhi Technological University

List of Experiments

Case Study:	Do the case study of basic Operating System commends.
Problem 1:	Write a Program to implement the First Come First Serve (FCFS) CPU Scheduling Algorithm.
Problem-2:	Write a Program to implement the Shortest Job First (SJF) CPU Scheduling Algorithm.
Problem 3:	Write a program to implement the Priority CPU Scheduling Algorithm.
Program 4:	Write a program to implement Round Robin Scheduling Algorithm.
Program 5:	Write a Program to implement Pre-emptive Shortest Job First Scheduling Algorithm.
Problem 6:	Write a Program to Implement the Longest Remaining Time First (LRTF) CPU Scheduling Algorithm
Program 7:	Write a Program to implement Bankers Algorithm.
Problem 8:	Write a program to implement the Producer-Consumer problem.
Problem 9:	Write a program to implement the First-In-First-Out (FIFO) Page Replacement Algorithm.
Problem 10:	Write a program to implement the Least Recently Used (LRU) Page Replacement Algorithm.
Problem 11:	Write a program to implement Dining-Philosophers Problem.
Problem 12:	Write a program to implement Reader Writer Problem.
Problem 13:	Write a program to implement the FCFS (First Come First Serve) Disk Scheduling Algorithms.
Problem 14:	Write a program to implement the SSTF (Shortest Seek Time First) Disk Scheduling Algorithms.
Problem 15:	Write a program to implement the SCAN Disk Scheduling Algorithms.

Case Study of Basic OS Commands

Aim : To study and perform basic operating system commands using the Windows Power Shell for file and directory management.

Theory : Windows is an operating system developed by Microsoft that provides command-line interfaces like Command Prompt (CMD) and PowerShell for interacting with the system using commands. These tools allow users to create files, navigate directories, view system information, and manage data efficiently.

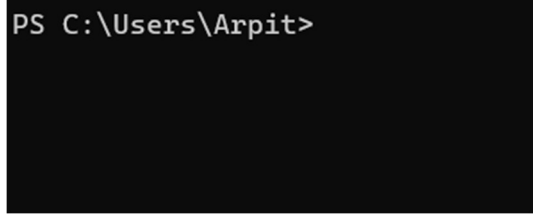
Environment Used :

- Operating System: Windows
- Interface: PowerShell

Case Study: Performing File Operations : We will create a folder called OS_Lab and perform various commands inside it.

Step 1: Open Terminal

Open:

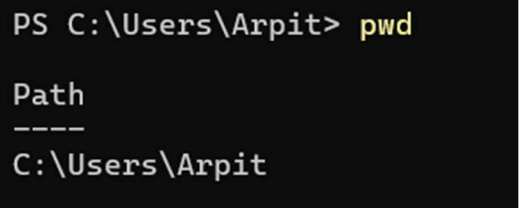


```
PS C:\Users\Arpit>
```

Step 2: Check Current Directory

- pwd

This shows the current working directory.



```
PS C:\Users\Arpit> pwd
Path
----
C:\Users\Arpit
```

Step 3: Create a New Folder

- mkdir OS_Lab

```
PS C:\Users\Arpit> mkdir OS_Lab

Directory: C:\Users\Arpit


Mode                LastWriteTime         Length Name
----                -
d-----         16-04-2026        22:30             OS_Lab
```

Step 4: Enter the Folder

- cd OS_Lab

```
PS C:\Users\Arpit> cd OS_Lab
PS C:\Users\Arpit\OS_Lab> |
```

Step 5: Create a File

- New-Item data.txt

```
PS C:\Users\Arpit\OS_Lab> New-Item data.txt

Directory: C:\Users\Arpit\OS_Lab


Mode                LastWriteTime         Length Name
----                -
-a-----         16-04-2026        22:32             0 data.txt
```

Step 6: Write Data to File

- echo "Operating System Lab" > data.txt

```
PS C:\Users\Arpit\OS_Lab> echo "Operating System Lab" > data.txt
PS C:\Users\Arpit\OS_Lab>
```

Step 7: Display File Content

- cat data.txt

```
PS C:\Users\Arpit\OS_Lab> cat data.txt
Operating System Lab
```

Step 8: Create Another File

- touch info.txt

```
Operating System Lab
PS C:\Users\Arpit\OS_Lab> New-Item info.txt

Directory: C:\Users\Arpit\OS_Lab

Mode                LastWriteTime         Length Name
----                -
-a-----         16-04-2026      22:34             0 info.txt
```

Step 9: Copy File

- cp data.txt info.txt

Step 10: Rename File

- mv info.txt details.txt

```
PS C:\Users\Arpit\OS_Lab> cp data.txt info.txt
PS C:\Users\Arpit\OS_Lab> mv info.txt details.txt
PS C:\Users\Arpit\OS_Lab> |
```

Step 11: List Files

- ls

```
PS C:\Users\Arpit\OS_Lab> mv info.txt details.txt
PS C:\Users\Arpit\OS_Lab> ls

Directory: C:\Users\Arpit\OS_Lab

Mode                LastWriteTime         Length Name
----                -
-a-----         16-04-2026      22:32             46 data.txt
-a-----         16-04-2026      22:32             46 details.txt
```

Step 12: Delete a File

- rm details.txt

```
PS C:\Users\Arpit\OS_Lab> rm details.txt
```

Observation Table :

Command	Function
pwd	Shows current directory
mkdir	Creates folder

cd	Changes directory
----	-------------------

touch	Creates file
-------	--------------

echo	Writes data
------	-------------

cat	Displays file
-----	---------------

cp	Copies file
----	-------------

mv	Renames file
----	--------------

ls	Lists files
----	-------------

rm	Deletes file
----	--------------

Conclusion :

The basic operating system commands were successfully executed using the macOS Terminal. These commands help users manage files, folders, and system operations efficiently.

Learning Outcomes :

- Learned to use Terminal in macOS
- Performed file and directory operations
- Understood UNIX-based OS commands
- Gained practical OS handling skills

Experiment No. 1

Implementation of First Come First Serve (FCFS) CPU Scheduling Algorithm

Aim :

To write a program to implement the **First Come First Serve (FCFS)** CPU scheduling algorithm and to calculate **Waiting Time**, **Turnaround Time**, and **Average Waiting Time** of processes.

Theory :

CPU Scheduling is a process used by the operating system to decide which process should get CPU time. One of the simplest CPU scheduling algorithms is **First Come First Serve (FCFS)**.

In FCFS scheduling:

- The process that arrives first gets the CPU first.
- It follows a **FIFO (First In First Out)** queue.
- Once a process starts executing, it continues until it finishes.

FCFS is:

- Easy to implement
- Non-preemptive
- Fair in arrival order

However, it may cause the **convoy effect**, where short processes wait for long processes to finish.

Algorithm (FCFS) :

1. Read number of processes n.
2. Read arrival time and burst time for each process.
3. Sort processes according to arrival time.
4. Set completion time of first process.
5. For each next process:
 - Start time = previous process completion time
 - Completion time = Start time + Burst time
6. Calculate Waiting Time and Turnaround Time.
7. Compute average values.
8. Display the result.

Code :

```
#include <stdio.h>
int main() {
    int n, i;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], wt[n], tat[n], ct[n];

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time of Process %d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    ct[0] = at[0] + bt[0];

    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    avg_wt /= n;
    avg_tat /= n;

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f", avg_tat);

    return 0;
}
```

Output :

```
Enter number of processes: 4
Enter Arrival Time and Burst Time of Process 1: 0 5
Enter Arrival Time and Burst Time of Process 2: 1 3
Enter Arrival Time and Burst Time of Process 3: 2 8
Enter Arrival Time and Burst Time of Process 4: 4 4

Process AT  BT  CT  TAT WT
P1  0   5   5   5   0
P2  1   3   8   7   4
P3  2   8  16  14   6
P4  4   4  20  16  12

Average Waiting Time = 5.50
Average Turnaround Time = 10.50
```

Conclusion :

The FCFS CPU scheduling algorithm was successfully implemented. The waiting time and turnaround time for each process were calculated, and average values were obtained. FCFS is simple but can lead to inefficiency when long processes arrive first.

Learning Outcomes :

- Understood the concept of CPU scheduling.
- Learned how FCFS algorithm works.
- Gained experience in calculating waiting time and turnaround time.
- Implemented a scheduling algorithm using C programming.

Experiment No. 2

Aim :

Write a Program to implement the Shortest Job First (SJF) CPU Scheduling

Algorithm :

Description: Associate with each process the length of its next CPU burst.

- Use these lengths to schedule the process with the shortest time.
- SJF is optimal – gives minimum average waiting time for a given set of processes.
- The difficulty is knowing the length of the next CPU request and Could ask the user.

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart:



- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$

Code:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int pid[n], bt[n], wt[n], tat[n];

    // Input burst time
    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        cout << "Enter burst time for Process " << pid[i] << ": ";
        cin >> bt[i];
    }
```

```

// Sorting processes according to burst time (SJF)
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (bt[i] > bt[j]) {
            swap(bt[i], bt[j]);
            swap(pid[i], pid[j]);
        }
    }
}

// Waiting time calculation
wt[0] = 0;
for (int i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time calculation
for (int i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

float avgWT = 0, avgTAT = 0;
for (int i = 0; i < n; i++) {
    avgWT += wt[i];
    avgTAT += tat[i];
}

avgWT /= n;
avgTAT /= n;

cout << "\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n";
for (int i = 0; i < n; i++) {
    cout << "P" << pid[i] << "\t\t"
        << bt[i] << "\t\t"
        << wt[i] << "\t\t"
        << tat[i] << endl;
}

cout << fixed << setprecision(2);
cout << "\nAverage Waiting Time = " << avgWT;
cout << "\nAverage Turnaround Time = " << avgTAT << endl;

return 0;
}

```

Output :

```
Enter number of processes: 4
Enter burst time for Process 1: 6
Enter burst time for Process 2: 8
Enter burst time for Process 3: 7
Enter burst time for Process 4: 3

Process Burst Time  Waiting Time  Turnaround Time
P4      3          0          3
P1      6          3          9
P3      7          9         16
P2      8         16         24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00
```

Conclusion :

The Shortest Job First (SJF) CPU Scheduling Algorithm was successfully implemented using C++. It schedules processes based on the shortest burst time, resulting in reduced average waiting and turnaround times. However, it may cause starvation for longer processes.

Learning Outcomes :

After this experiment, the student can:

- Understand the concept of SJF CPU scheduling
- Implement SJF using C++
- Calculate waiting time and turnaround time
- Analyze the efficiency and limitation of SJF

Experiment No. 3

Aim :

Write a program to implement the Priority CPU Scheduling Algorithm.

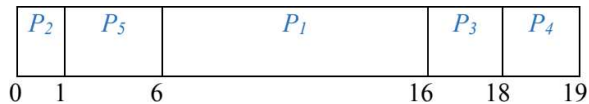
Description:

A priority number (integer) is associated with each process

- The CPU is allocated to the process with the highest priority (smallest integer highest priority)
 - Preemptive
 - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem Starvation – low priority processes may never execute
- Solution Aging – as time progresses increase the priority of the process

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec

Code:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int pid[n], bt[n], pr[n], wt[n], tat[n];
```

```

// Input burst time and priority
for (int i = 0; i < n; i++) {
    pid[i] = i + 1;
    cout << "Enter burst time for Process " << pid[i] << ": ";
    cin >> bt[i];
    cout << "Enter priority for Process " << pid[i] << ": ";
    cin >> pr[i];
}

// Sort processes according to priority
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (pr[i] > pr[j]) {
            swap(pr[i], pr[j]);
            swap(bt[i], bt[j]);
            swap(pid[i], pid[j]);
        }
    }
}

// Waiting time calculation
wt[0] = 0;
for (int i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time calculation
for (int i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

float avgWT = 0, avgTAT = 0;
for (int i = 0; i < n; i++) {
    avgWT += wt[i];
    avgTAT += tat[i];
}

avgWT /= n;
avgTAT /= n;

cout << "\nProcess\tBurst Time\tPriority\tWaiting Time\tTurnaround Time\n";
for (int i = 0; i < n; i++) {
    cout << "P" << pid[i] << "\t\t"

```

```

        << bt[i] << "\t\t"
        << pr[i] << "\t\t"
        << wt[i] << "\t\t"
        << tat[i] << endl;
    }

    cout << fixed << setprecision(2);
    cout << "\nAverage Waiting Time = " << avgWT;
    cout << "\nAverage Turnaround Time = " << avgTAT << endl;

    return 0;
}

```

Output :

```

Enter number of processes: 5
Enter burst time for Process 1: 10
Enter priority for Process 1: 3
Enter burst time for Process 2: 1
Enter priority for Process 2: 1
Enter burst time for Process 3: 2
Enter priority for Process 3: 4
Enter burst time for Process 4: 1
Enter priority for Process 4: 5
Enter burst time for Process 5: 5
Enter priority for Process 5: 2

```

Process	Burst Time	Priority	Waiting Time	Turnaround Time
P2	1	1	0	1
P5	5	2	1	6
P1	10	3	6	16
P3	2	4	16	18
P4	1	5	18	19

```

Average Waiting Time = 8.20
Average Turnaround Time = 12.00

```

Conclusion :

The Priority CPU Scheduling Algorithm was successfully implemented using C++. Processes were scheduled based on their priority, ensuring higher-priority processes execute first. While the algorithm improves responsiveness for critical tasks, it may cause starvation for low-priority processes.

Learning Outcomes :

After this experiment, the student can:

- Understand Priority CPU scheduling
- Implement Priority scheduling using C++
- Compute waiting time and turnaround time
- Identify advantages and drawbacks such as starvation

EXPERIMENT - 4

Aim :

To implement the Round Robin CPU Scheduling Algorithm and analyze process execution based on a fixed time quantum, calculating Waiting Time and Turnaround Time for each process.

Theory :

Round Robin (RR) is a preemptive CPU scheduling algorithm designed mainly for time-sharing systems.

- Each process is assigned a small unit of CPU time called Time Quantum (q).
- Processes are stored in a ready queue (FIFO order).
- The CPU is given to the first process for at most q units of time.
- If the process finishes before q, it leaves the system.
- Otherwise, it is preempted and placed at the end of the ready queue.
- This cycle repeats until all processes complete.

Key Terms

- Burst Time (BT): Total CPU time required by a process.
- Waiting Time (WT): Total time a process waits in the ready queue.
- Turnaround Time (TAT): Time from arrival to completion.

Advantages

- Fair to all processes (no starvation).
- Good response time for interactive systems.

Disadvantages

- Performance heavily depends on the chosen time quantum.
 - Too small → too many context switches.
 - Too large → behaves like FCFS.

Code :

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<int> burst(n), remaining(n);
    vector<string> process(n);

    for(int i = 0; i < n; i++) {
        cout << "Enter Process ID and Burst Time for P" << i+1 << ": ";
        cin >> process[i] >> burst[i];
        remaining[i] = burst[i];
    }

    int quantum;
    cout << "Enter Time Quantum: ";
    cin >> quantum;

    vector<int> waiting(n, 0), turnaround(n, 0);

    int time = 0;

    while(true) {
        bool done = true;

        for(int i = 0; i < n; i++) {
            if(remaining[i] > 0) {
                done = false;

                if(remaining[i] > quantum) {
                    time += quantum;
                    remaining[i] -= quantum;
                } else {
                    time += remaining[i];
                    waiting[i] = time - burst[i];
                    remaining[i] = 0;
                }
            }
        }

        if(done)
            break;
    }

    for(int i = 0; i < n; i++) {
        turnaround[i] = burst[i] + waiting[i];
    }
}
```

```

}

cout << "\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n";
for(int i = 0; i < n; i++) {
    cout << process[i] << "\t\t"
        << burst[i] << "\t\t"
        << waiting[i] << "\t\t"
        << turnaround[i] << endl;
}

double avgWT = 0, avgTAT = 0;
for(int i = 0; i < n; i++) {
    avgWT += waiting[i];
    avgTAT += turnaround[i];
}

avgWT /= n;
avgTAT /= n;

cout << fixed << setprecision(2);
cout << "\nAverage Waiting Time = " << avgWT;
cout << "\nAverage Turnaround Time = " << avgTAT << endl;

return 0;
}

```

Output :

```

Enter number of processes: 4
Enter Process ID and Burst Time for P1: P1 4
Enter Process ID and Burst Time for P2: P2 3
Enter Process ID and Burst Time for P3: P3 6
Enter Process ID and Burst Time for P4: P4 2
Enter Time Quantum: 2

Process Burst Time  Waiting Time  Turnaround Time
P1      4           6           10
P2      3           8           11
P3      6           9           15
P4      2           6           8

Average Waiting Time = 7.25
Average Turnaround Time = 11.00

```

Conclusion :

In this experiment, the Round Robin CPU Scheduling Algorithm was successfully implemented.

We observed that:

- Each process received CPU time fairly using a fixed time quantum.
- No process suffered starvation.
- Waiting Time and Turnaround Time depend strongly on the chosen quantum.

Thus, Round Robin is well-suited for time-sharing and interactive systems, provided an appropriate time quantum is selected.

EXPERIMENT - 5

Aim :

To implement the Shortest Remaining Time First (SRTF) CPU Scheduling Algorithm and compute Waiting Time and Turnaround Time for each process.

Theory :

Shortest Remaining Time First (SRTF) is the preemptive version of Shortest Job First (SJF).

- At every unit of time, the CPU is assigned to the process with the smallest remaining burst time.
- If a new process arrives with a shorter remaining time than the currently running process, the CPU is preempted and reassigned.
- This continues until all processes finish execution.

Key Terms

- Arrival Time (AT): Time at which a process enters the ready queue.
- Burst Time (BT): Total CPU time required by a process.
- Remaining Time (RT): CPU time still needed to complete execution.
- Waiting Time (WT): Total time spent waiting in the ready queue.
- Turnaround Time (TAT):

Advantages

- Produces minimum average waiting time (optimal among scheduling algorithms).
- Very responsive to short jobs.

Disadvantages

- Requires knowing burst times in advance.
- Can cause starvation of long processes.
- Higher overhead due to frequent context switching.

Code :

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<int> at(n), bt(n), rt(n), ct(n), wt(n), tat(n);
    vector<string> pid(n);

    for(int i = 0; i < n; i++) {
        cout << "Enter Process ID, Arrival Time and Burst Time: ";
        cin >> pid[i] >> at[i] >> bt[i];
        rt[i] = bt[i];
    }

    int completed = 0, time = 0;
    int min_rt, shortest = -1;
    bool found = false;

    while(completed != n) {
        min_rt = INT_MAX;
        found = false;

        for(int i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && rt[i] < min_rt) {
                min_rt = rt[i];
                shortest = i;
                found = true;
            }
        }

        if(!found) {
            time++;
            continue;
        }

        // Execute for 1 unit of time
        rt[shortest]--;
        time++;
    }
}
```

```

// If process completes
if(rt[shortest] == 0) {
    completed++;
    ct[shortest] = time;
    tat[shortest] = ct[shortest] - at[shortest];
    wt[shortest] = tat[shortest] - bt[shortest];
}
}

double avgWT = 0, avgTAT = 0;

cout << "\nProcess\tAT\tBT\tWT\tTAT\n";
for(int i = 0; i < n; i++) {
    cout << pid[i] << "\t"
        << at[i] << "\t"
        << bt[i] << "\t"
        << wt[i] << "\t"
        << tat[i] << endl;

    avgWT += wt[i];
    avgTAT += tat[i];
}

avgWT /= n;
avgTAT /= n;

cout << fixed << setprecision(2);
cout << "\nAverage Waiting Time = " << avgWT;
cout << "\nAverage Turnaround Time = " << avgTAT << endl;

return 0;
}

```

Output :

```
Enter number of processes: 4
Enter Process ID, Arrival Time and Burst Time: P1 0 3
Enter Process ID, Arrival Time and Burst Time: P2 4 2
Enter Process ID, Arrival Time and Burst Time: P3 6 4
Enter Process ID, Arrival Time and Burst Time: P4 7 1

Process AT  BT  WT  TAT
P1  0  3  0  3
P2  4  2  0  2
P3  6  4  1  5
P4  7  1  0  1

Average Waiting Time = 0.25
Average Turnaround Time = 2.75
```

Conclusion :

In this experiment, the Shortest Remaining Time First (SRTF) algorithm was successfully implemented. The scheduler always selected the process with the least remaining burst time, preempting the current process when necessary.

We observed that:

- Short processes complete very quickly.
- Average waiting time is minimized.
- Longer processes may suffer starvation.

Hence, SRTF is efficient for minimizing waiting time but is less suitable when fairness is required.

EXPERIMENT - 6

Aim :

To implement the Longest Remaining Time First (LRTF) CPU Scheduling Algorithm and calculate Waiting Time and Turnaround Time for each process.

Theory :

Longest Remaining Time First (LRTF) is the preemptive variant of Longest Job First (LJF).

- At every unit of time, the CPU is assigned to the process having the maximum remaining burst time.
- Whenever a new process arrives with longer remaining time than the currently running process, preemption occurs.
- Execution continues until all processes complete.

Important Terms

- Arrival Time (AT): Time when a process enters the ready queue.
- Burst Time (BT): Total CPU time required.
- Remaining Time (RT): CPU time still needed.
- Completion Time (CT): Time at which a process finishes.
- Turnaround Time (TAT):
- Waiting Time (WT):

Characteristics

- Preemptive in nature.
- Favors longer jobs over shorter ones.

Advantages

- Useful in systems where long tasks are prioritized.

Disadvantages

- Short processes may suffer starvation.
- Average waiting time is usually high.
- Requires frequent comparisons → higher overhead.

Code :

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<string> pid(n);
    vector<int> at(n), bt(n), rt(n), ct(n), wt(n), tat(n);

    for(int i = 0; i < n; i++) {
        cout << "Enter Process ID, Arrival Time and Burst Time: ";
        cin >> pid[i] >> at[i] >> bt[i];
        rt[i] = bt[i];
    }

    int completed = 0, time = 0;

    while(completed != n) {
        int longest = -1;
        int max_rt = -1;

        // Find process with maximum remaining time at current time
        for(int i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && rt[i] > max_rt) {
                max_rt = rt[i];
                longest = i;
            }
        }

        // If no process has arrived yet
        if(longest == -1) {
            time++;
            continue;
        }

        // Execute selected process for 1 unit time
        rt[longest]--;
        time++;

        // If process finishes
        if(rt[longest] == 0) {
```

```

        completed++;
        ct[longest] = time;
        tat[longest] = ct[longest] - at[longest];
        wt[longest] = tat[longest] - bt[longest];
    }
}

double avgWT = 0, avgTAT = 0;

cout << "\nProcess\tAT\tBT\tWT\tTAT\n";
for(int i = 0; i < n; i++) {
    cout << pid[i] << "\t"
        << at[i] << "\t"
        << bt[i] << "\t"
        << wt[i] << "\t"
        << tat[i] << endl;

    avgWT += wt[i];
    avgTAT += tat[i];
}

avgWT /= n;
avgTAT /= n;

cout << fixed << setprecision(2);
cout << "\nAverage Waiting Time = " << avgWT;
cout << "\nAverage Turnaround Time = " << avgTAT << endl;

return 0;
}

```

Output :

```
Enter number of processes: 4
Enter Process ID, Arrival Time and Burst Time: P1 0 2
Enter Process ID, Arrival Time and Burst Time: P2 4 3
Enter Process ID, Arrival Time and Burst Time: P3 6 7
Enter Process ID, Arrival Time and Burst Time: P4 8 3

Process AT  BT  WT  TAT
P1  0  2  0  2
P2  4  3  8  11
P3  6  7  3  10
P4  8  3  6  9

Average Waiting Time = 4.25
Average Turnaround Time = 8.00
```

Conclusion :

In this experiment, the Longest Remaining Time First (LRTF) scheduling algorithm was successfully implemented.

Observations:

- The CPU always selected the process with the largest remaining burst time.
- Preemption occurred whenever a longer job arrived.
- Long processes were favored, while short jobs experienced longer waiting times.

Hence, LRTF is suitable only in special scenarios where long-running tasks are prioritized, but it is generally inefficient for minimizing average waiting time.

EXPERIMENT - 7

Aim :

Implement HRRN Algorithm

Theory :

HRRN is a non-preemptive scheduling algorithm that selects the process having the highest response ratio

The response ratio is calculated as:

$$\text{Response Ratio} = \frac{\text{Waiting Time} + \text{Burst Time}}{\text{Burst Time}}$$

Key idea:

- Long waiting processes gradually get higher priority.
- Prevents starvation.
- Balances short and long jobs.

Steps (HRRN)

1. Read process details (Arrival Time, Burst Time).
2. At each scheduling point, compute Response Ratio for all ready processes.
3. Select the process with maximum Response Ratio.
4. Execute it completely (non-preemptive).
5. Repeat until all processes finish.

Algorithm :

1. Input number of processes n.
2. Input Arrival Time and Burst Time for each process.
3. Initialize current time = 0.
4. While not all processes completed:
 - For every ready process, calculate:
 $RR = (WT + BT) / BT$
 - Select process with maximum RR.
 - Execute selected process fully.
 - Update Waiting Time and Turnaround Time.
5. Compute average Waiting Time and Turnaround Time.
6. Display Gantt Chart and results.

Code :

```
#include <bits/stdc++.h>
using namespace std;
struct Process {
    int pid;
    int at;    // Arrival Time
    int bt;    // Burst Time
    int ct;    // Completion Time
    int tat;   // Turnaround Time
    int wt;    // Waiting Time
    bool done = false;
};
int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    vector<Process> p(n);
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        cout << "Enter Arrival Time and Burst Time for P" << i + 1 << ": ";
        cin >> p[i].at >> p[i].bt;
    }

    int completed = 0;
    int currentTime = 0;
    while (completed < n) {
        double maxRR = -1.0;
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (!p[i].done && p[i].at <= currentTime) {
                int wt = currentTime - p[i].at;
                double rr = (double)(wt + p[i].bt) / p[i].bt;
                if (rr > maxRR) {
```

```

        maxRR = rr;
        idx = i;
    }
}
// If no process has arrived yet
if (idx == -1) {
    currentTime++;
    continue;
}
// Execute selected process fully
currentTime += p[idx].bt;
p[idx].ct = currentTime;
p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;
p[idx].done = true;
completed++;
}
double avgWT = 0, avgTAT = 0;
cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
for (int i = 0; i < n; i++) {
    cout << "P" << p[i].pid << "\t" << p[i].at << "\t" << p[i].bt << "\t"
        << p[i].ct << "\t" << p[i].tat << "\t" << p[i].wt << "\n";
    avgWT += p[i].wt;
    avgTAT += p[i].tat;
}
cout << "\nAverage Waiting Time = " << avgWT / n;
cout << "\nAverage Turnaround Time = " << avgTAT / n << endl;
return 0;
}

```

Output :

```
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 4 2
Enter Arrival Time and Burst Time for P3: 7 2
Enter Arrival Time and Burst Time for P4: 8 9
```

PID	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	4	2	6	2	0
P3	7	2	9	2	0
P4	8	9	18	10	1

Average Waiting Time = 0.25

Average Turnaround Time = 4.25

EXPERIMENT - 8

Aim : Implement Improved Round Robin Algorithm

Theory :

Improved Round Robin is an enhancement of traditional Round Robin where the time quantum is adjusted dynamically (often using average burst time or remaining burst time).

Goals:

- Reduce excessive context switches
- Improve average waiting time
- Give shorter jobs faster completion
- Still maintain fairness

Typical improvement approach:

- Calculate dynamic time quantum (example: average of remaining burst times).
- Sort or prioritize shorter remaining jobs when possible.
- Execute processes with updated quantum.

Steps (Improved RR – generic)

1. Read process details.
2. Insert processes into ready queue.
3. Compute Dynamic Time Quantum (DTQ).
4. Assign CPU for min (DTQ, Remaining Burst).
5. Update remaining burst time.
6. If unfinished, push back to queue .
7. Recalculate DTQ periodically.
8. Continue until all processes complete.

Algorithm :

1. Input process details.
2. Add all processes to ready queue.
3. While queue is not empty:
 - Compute Dynamic Time Quantum (average of remaining burst times).
 - Pick next process.
 - Execute for min(DTQ, remaining burst).
 - Update remaining burst.
 - If remaining > 0, reinsert into queue.
4. After completion, calculate WT and TAT.
5. Display results.

Code :

```
#include <bits/stdc++.h>
using namespace std;

struct Process {
    int pid;
    int at;
    int bt;
    int rem;
    int ct = 0;
};

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    vector<Process> p(n);
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        cout << "Enter Arrival Time and Burst Time for P" << i + 1 << ": ";
        cin >> p[i].at >> p[i].bt;
        p[i].rem = p[i].bt;
    }
    queue<int> q;
    int time = 0, completed = 0;
    vector<bool> inQueue(n, false);

    // Sort by arrival time
    sort(p.begin(), p.end(), [](auto &a, auto &b) {
        return a.at < b.at;
    });
    q.push(0);
    inQueue[0] = true;
```

```

time = p[0].at;
while (completed < n) {
    if (q.empty()) {
        for (int i = 0; i < n; i++) {
            if (p[i].rem > 0) {
                q.push(i);
                inQueue[i] = true;
                time = max(time, p[i].at);
                break;
            }
        }
    }
    // Calculate Dynamic Time Quantum (average remaining burst)
    int sum = 0, cnt = 0;
    for (int i = 0; i < n; i++) {
        if (p[i].rem > 0) {
            sum += p[i].rem;
            cnt++;
        }
    }
    int tq = (int)ceil((double)sum / cnt);
    int i = q.front();
    q.pop();
    inQueue[i] = false;
    int exec = min(tq, p[i].rem);
    time += exec;
    p[i].rem -= exec;
    // Add newly arrived processes
    for (int j = 0; j < n; j++) {
        if (p[j].rem > 0 && !inQueue[j] && p[j].at <= time) {
            q.push(j);
            inQueue[j] = true;
        }
    }
    if (p[i].rem > 0) {
        q.push(i);
    }
}

```

```

        inQueue[i] = true;
    } else {
        p[i].ct = time;
        completed++;
    }
}

double avgWT = 0, avgTAT = 0;
cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
for (int i = 0; i < n; i++) {
    int tat = p[i].ct - p[i].at;
    int wt = tat - p[i].bt;
    cout << "P" << p[i].pid << "\t" << p[i].at << "\t" << p[i].bt << "\t"
        << p[i].ct << "\t" << tat << "\t" << wt << "\n";
    avgWT += wt;
    avgTAT += tat;
}
cout << "\nAverage Waiting Time = " << avgWT / n;
cout << "\nAverage Turnaround Time = " << avgTAT / n << endl;

return 0;
}

```

Output :

```
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 4 2
Enter Arrival Time and Burst Time for P3: 7 2
Enter Arrival Time and Burst Time for P4: 8 9
```

PID	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	4	2	6	2	0
P3	7	2	9	2	0
P4	8	9	18	10	1

Average Waiting Time = 0.25

Average Turnaround Time = 4.25

EXPERIMENT 9

Aim :

To implement the Longest Remaining Time First (LRTF) CPU scheduling algorithm and calculate waiting time and turnaround time.

Description :

Longest Remaining Time First (LRTF) is a preemptive CPU scheduling algorithm in which the process having the largest remaining burst time is selected for execution.

If a new process arrives with a longer remaining time, the current process is preempted.

It is the opposite of Shortest Remaining Time First (SRTF).

Code :

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<int> bt(n), rt(n), wt(n, 0), tat(n);

    cout << "Enter burst times:\n";
    for(int i = 0; i < n; i++) {
        cin >> bt[i];
        rt[i] = bt[i];
    }

    int time = 0, completed = 0;

    while(completed < n) {
        int max_index = -1;
        int max_remaining = -1;

        // Find process with maximum remaining time
        for(int i = 0; i < n; i++) {
            if(rt[i] > max_remaining) {
```

```

        if(rt[i] > 0) {
            max_remaining = rt[i];
            max_index = i;
        }
    }

    if(max_index == -1)
        break;

    rt[max_index]--;
    time++;

    if(rt[max_index] == 0) {
        completed++;
        tat[max_index] = time;
        wt[max_index] = tat[max_index] - bt[max_index];
    }
}

float total_wt = 0, total_tat = 0;

cout << "\nProcess\tBT\tWT\tTAT\n";
for(int i = 0; i < n; i++) {
    total_wt += wt[i];
    total_tat += tat[i];
    cout << i+1 << "\t" << bt[i] << "\t"
        << wt[i] << "\t" << tat[i] << endl;
}

cout << "\nAverage Waiting Time: " << total_wt/n;
cout << "\nAverage Turnaround Time: " << total_tat/n;

return 0;
}

```

Output :

```
Enter number of processes: 4
Enter burst times:
3 5 7 4
```

Process	BT	WT	TAT
1	3	13	16
2	5	12	17
3	7	11	18
4	4	15	19

```
Average Waiting Time: 12.75
Average Turnaround Time: 17.5
```

Conclusion :

Thus, the Longest Remaining Time First scheduling algorithm was successfully implemented. It favors processes with larger burst times but may increase waiting time for shorter processes.

EXPERIMENT 10

Aim :

To implement the Producer-Consumer problem using semaphores in C++.

Description :

The Producer-Consumer problem is a classic synchronization problem where:

- Producer produces data and adds it to the buffer.
- Consumer consumes data from the buffer.
- Semaphores are used to avoid race conditions.

It uses:

- mutex → for mutual exclusion
- empty → counts empty buffer slots
- full → counts filled buffer slots

Code :

```
#include <iostream>
#include <thread>
#include <mutex>
#include <semaphore.h>

using namespace std;

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty_slots, full_slots;
mutex mtx;

void producer() {
    for(int i = 1; i <= 10; i++) {
        sem_wait(&empty_slots);

        mtx.lock();
        buffer[in] = i;
```

```

        cout << "Produced: " << i << endl;
        in = (in + 1) % BUFFER_SIZE;
        mtx.unlock();

        sem_post(&full_slots);
    }
}

void consumer() {
    for(int i = 1; i <= 10; i++) {
        sem_wait(&full_slots);

        mtx.lock();
        int item = buffer[out];
        cout << "Consumed: " << item << endl;
        out = (out + 1) % BUFFER_SIZE;
        mtx.unlock();

        sem_post(&empty_slots);
    }
}

int main() {

    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);

    thread t1(producer);
    thread t2(consumer);

    t1.join();
    t2.join();

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);

    return 0;
}

```

Output :

```
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Consumed: 6
Consumed: 7
Consumed: 8
Produced: 9
Produced: 10
Consumed: 9
Consumed: 10
```

Conclusion :

Thus, the Producer-Consumer problem was successfully implemented using semaphores and mutex. Synchronization prevents race conditions and ensures proper resource sharing.

EXPERIMENT 11

Aim :

To implement the Dining Philosopher problem using threads and mutex in C++.

Description :

The Dining Philosopher problem illustrates synchronization issues:

- 5 philosophers sit around a table.
- Each philosopher needs 2 forks to eat.
- Deadlock may occur if all philosophers pick one fork simultaneously.
- Mutex locks are used to prevent deadlock.

Code :

```
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

using namespace std;

#define N 5

mutex forks[N];

void philosopher(int id) {

    for(int i = 0; i < 3; i++) {

        cout << "Philosopher " << id << " is thinking\n";
        this_thread::sleep_for(chrono::milliseconds(500));

        if(id == N - 1) {
            // Last philosopher picks right fork first (avoid deadlock)
            forks[(id + 1) % N].lock();
            forks[id].lock();
        } else {
            forks[id].lock();
            forks[(id + 1) % N].lock();
        }
    }
}
```

```

    }

    cout << "Philosopher " << id << " is eating\n";
    this_thread::sleep_for(chrono::milliseconds(500));

    forks[id].unlock();
    forks[(id + 1) % N].unlock();
}
}

int main() {

    thread philosophers[N];

    for(int i = 0; i < N; i++) {
        philosophers[i] = thread(philosopher, i);
    }

    for(int i = 0; i < N; i++) {
        philosophers[i].join();
    }

    return 0;
}

```

Output :

```
Philosopher 0 is thinking
Philosopher Philosopher 2 is thinking
1Philosopher 3 is thinking
  is thinking
Philosopher 4 is thinking
Philosopher 2 is eating
Philosopher 0 is eating
Philosopher 2 is thinking
Philosopher 0 is thinking
Philosopher Philosopher 3 is eating
1 is eating
Philosopher 3 is thinking
Philosopher 2 is eating
Philosopher 4 is eating
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 1 is eating
Philosopher 4 is thinking
Philosopher 3 is eating
Philosopher 3 is thinking
Philosopher 0 is eating
Philosopher 2 is eating
Philosopher 1 is thinking
Philosopher 0 is thinking
Philosopher 4 is eating
Philosopher 1 is eating
Philosopher 4 is thinking
Philosopher 3 is eating
Philosopher 0 is eating
Philosopher 4 is eating
```

Conclusion :

Thus, the Dining Philosopher problem was successfully implemented using mutex locks. Proper synchronization prevents deadlock and ensures safe resource allocation.

Experiment 12

Aim : To implement a solution for the Cigarette Smoker's Problem using synchronization techniques to avoid deadlock and ensure proper resource sharing among processes.

Description : The Cigarette Smoker's Problem is a classic synchronization problem in Operating Systems that demonstrates process coordination using semaphores.

Problem Statement:

- There are 3 smokers and 1 agent.
- Each smoker has one of the following:
 - Smoker 1 → Tobacco
 - Smoker 2 → Paper
 - Smoker 3 → Matches
- The agent randomly places two of the three items on the table.
- The smoker who has the third missing item picks up the items, makes a cigarette, and smokes.

Challenges:

- Avoid deadlock
- Ensure mutual exclusion
- Proper synchronization between agent and smokers

Solution Approach:

- Use semaphores:
 - agentSem → Controls agent execution
 - tobacco, paper, match → Signals smokers
- Only one smoker acts at a time.
- After smoking, the smoker signals the agent to continue.

Code :

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <cstdlib>
```

```

#include <ctime>
using namespace std;

mutex mtx;
condition_variable cv;
bool tobacco = false, paper = false, matchstick = false;
bool ready = false;
int ITERATIONS = 10;
// Agent function
void agent() {
    for (int i = 0; i < ITERATIONS; i++) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [] { return !ready; });

        int choice = rand() % 3;

        if (choice == 0) {
            cout << "\nAgent puts Tobacco + Paper\n";
            tobacco = true;
            paper = true;
        }
        else if (choice == 1) {
            cout << "\nAgent puts Paper + Match\n";
            paper = true;
            matchstick = true;
        }
        else {
            cout << "\nAgent puts Tobacco + Match\n";
            tobacco = true;
            matchstick = true;
        }
        ready = true;
        cv.notify_all();
    }
}

// Smoker with Tobacco
void smoker_tobacco() {
    for (int i = 0; i < ITERATIONS; i++) {
        unique_lock<mutex> lock(mtx);
    }
}

```

```

        cv.wait(lock, [] { return ready && paper && matchstick; });
        cout << "Smoker with Tobacco smokes\n";
        paper = false;
        matchstick = false;
        ready = false;
        cv.notify_all();
    }
}

// Smoker with Paper
void smoker_paper() {
    for (int i = 0; i < ITERATIONS; i++) {
        unique_lock<mutex> lock(mtx);
        cv.wait(lock, [] { return ready && tobacco && matchstick; });
        cout << "Smoker with Paper smokes\n";
        tobacco = false;
        matchstick = false;
        ready = false;

        cv.notify_all();
    }
}

// Smoker with Match
void smoker_match() {
    for (int i = 0; i < ITERATIONS; i++) {
        unique_lock<mutex> lock(mtx);
        cv.wait(lock, [] { return ready && tobacco && paper; });
        cout << "Smoker with Match smokes\n";
        tobacco = false;
        paper = false;
        ready = false;
        cv.notify_all();
    }
}

int main() {
    srand(time(0));
    thread t1(agent);
    thread t2(smoker_tobacco);
    thread t3(smoker_paper);

```

```
    thread t4(smoker_match);  
    t1.join();  
    t2.join();  
    t3.join();  
    t4.join();  
    cout << "\nExecution Finished Successfully\n";  
    return 0;  
}
```

Output :

```
Agent puts Tobacco + Match  
Smoker with Paper smokes  
  
Agent puts Tobacco + Paper  
Smoker with Match smokes  
  
Agent puts Tobacco + Paper  
Smoker with Match smokes  
  
Agent puts Tobacco + Match  
Smoker with Paper smokes
```

Conclusion :

- The Cigarette Smoker's Problem demonstrates process synchronization using semaphores.
- It ensures:
 - No deadlock
 - Proper resource allocation
 - Efficient inter-process communication
- This experiment highlights the importance of synchronization in multi-threaded environments and helps understand real-world OS scheduling problems.

Experiment 13

Aim : To implement a solution for the Sleeping Barber Problem using synchronization techniques (semaphores) to manage customer and barber processes efficiently.

Description : The Sleeping Barber Problem is a classical synchronization problem that illustrates process coordination, mutual exclusion, and resource management.

Problem Statement:

- A barber shop has:
 - 1 barber
 - 1 barber chair
 - N waiting chairs
- Behavior:
 - If no customers → barber sleeps
 - If a customer arrives:
 - If barber is sleeping → wake him up
 - If chairs are available → wait
 - If no chairs → leave

Challenges:

- Avoid race conditions
- Ensure mutual exclusion
- Prevent lost wakeups

Solution Approach:

- customers → counts waiting customers
- barber → indicates barber availability
- mutex → ensures mutual exclusion

Code :

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <chrono>
```

```

using namespace std;
mutex mtx;
condition_variable cv_barber, cv_customer;
int waiting_chairs = 3;
queue<int> customers;
bool done = false;

// Barber function
void barber() {
    while (!done) {
        unique_lock<mutex> lock(mtx);
        // Sleep if no customers
        cv_barber.wait(lock, [] { return !customers.empty() || done; });
        if (done && customers.empty()) break;
        int customer_id = customers.front();
        customers.pop();
        cout << "Barber is cutting hair of Customer " << customer_id << endl;
        lock.unlock();
        // Simulate haircut
        this_thread::sleep_for(chrono::seconds(1));
        cout << "Customer " << customer_id << " is done\n";
        cv_customer.notify_all();
    }
    cout << "Barber shop closing...\n";
}

// Customer function
void customer(int id) {
    unique_lock<mutex> lock(mtx);

    if (customers.size() < waiting_chairs) {
        cout << "Customer " << id << " is waiting\n";
        customers.push(id);
        cv_barber.notify_one(); // wake barber
    } else {
        cout << "Customer " << id << " leaves (no chair available)\n";
    }
}

```

```

int main() {
    thread b(barber);

    // Create customers
    for (int i = 1; i <= 10; i++) {
        thread(customer, i).detach();
        this_thread::sleep_for(chrono::milliseconds(300));
    }
    // Wait for all customers to be processed
    this_thread::sleep_for(chrono::seconds(5))
    done = true;
    cv_barber.notify_one();
    b.join();
    cout << "Execution finished\n";
    return 0;
}

```

Output :

```

Customer 1 is waiting
Barber is cutting hair of Customer 1
Customer 2 is waiting
Customer 3 is waiting
Customer 4 is waiting
Customer 1 is done
Barber is cutting hair of Customer 2
Customer 5 is waiting
Customer 6 leaves (no chair available)
Customer 7 leaves (no chair available)
Customer 2 is done
Barber is cutting hair of Customer 3
Customer 8 is waiting
Customer 9 leaves (no chair available)
Customer 10 leaves (no chair available)
Customer 3 is done
Barber is cutting hair of Customer 4
Customer 4 is done
Barber is cutting hair of Customer 5
Customer 5 is done
Barber is cutting hair of Customer 8
Customer 8 is done
Barber shop closing...

```

Conclusion :

- The Sleeping Barber Problem demonstrates process synchronization using semaphores.
- It effectively handles:
 - Concurrent access
 - Limited resources (chairs)
 - Process coordination
- The solution avoids:
 - Deadlock
 - Race conditions
 - Starvation (in controlled scenarios)

Experiment 14

Aim : To implement a solution to the Readers-Writers Problem using synchronization techniques (semaphores) to manage concurrent access to shared data.

Description : The Readers-Writers Problem is a classic synchronization problem that deals with multiple processes accessing shared data.

Problem Statement: There are two types of processes:

- Readers → Only read the data
- Writers → Modify the data

Solution Approach:

We use semaphores:

- mutex → protects readCount variable
- wrt → allows exclusive access for writers
- readCount → keeps track of active readers

Working:

- First reader locks the resource
- Last reader releases it
- Writers must wait until no reader is active

Code :

```
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>
using namespace std;
mutex mtx;    // protects read_count
mutex wrt;    // controls writer access
```

```
int read_count = 0;
int shared_data = 0;
```

```
// Reader function
void reader(int id) {
```

```

for (int i = 0; i < 2; i++) {
    // Entry section
    mtx.lock();
    read_count++;
    if (read_count == 1) {
        wrt.lock(); // first reader blocks writer
    }
    mtx.unlock();
    // Critical section (reading)
    cout << "Reader " << id << " is reading: " << shared_data << endl;
    this_thread::sleep_for(chrono::milliseconds(500));
    // Exit section
    mtx.lock();
    read_count--;
    if (read_count == 0) {
        wrt.unlock(); // last reader allows writer
    }
    mtx.unlock();
    this_thread::sleep_for(chrono::milliseconds(300));
}
}

// Writer function
void writer(int id) {
    for (int i = 0; i < 2; i++) {
        wrt.lock(); // exclusive access

        shared_data++;
        cout << "Writer " << id << " is writing: " << shared_data << endl;
        this_thread::sleep_for(chrono::milliseconds(700));
        wrt.unlock();
        this_thread::sleep_for(chrono::milliseconds(500));
    }
}

int main() {
    thread r1(reader, 1);
    thread r2(reader, 2);
    thread r3(reader, 3);
    thread w1(writer, 1);
    thread w2(writer, 2);
}

```

```
    r1.join();
    r2.join();
    r3.join();
    w1.join();
    w2.join();
    cout << "\nExecution Finished\n";
    return 0;
}
```

Output :

```
Reader 1 is reading: 0
Reader 2 is reading: 0
Reader 3 is reading: 0
Writer 1 is writing: 1
Writer 2 is writing: 2
Reader 2 is reading: 2
Reader 1 is reading: 2
Reader 3 is reading: 2
Writer 1 is writing: 3
Writer 2 is writing: 4
```

Conclusion :

- The Readers-Writers Problem demonstrates controlled access to shared resources using semaphores.
- It ensures:
 - Multiple readers can read simultaneously
 - Writers get exclusive access
- The implementation prevents:
 - Race conditions
 - Data inconsistency

Experiment 15

Aim : To implement the Banker's Algorithm for deadlock avoidance and determine whether the system is in a safe state.

Description : The Banker's Algorithm is a deadlock avoidance algorithm used in Operating Systems. It ensures that resource allocation is done in such a way that the system always remains in a safe state.

Key Concepts:

- Available → Number of available resources
- Max → Maximum demand of each process
- Allocation → Resources currently allocated
- Need → Remaining resources required

Working Principle:

1. Calculate the Need matrix
2. Find a process whose:
 - $\text{Need} \leq \text{Available}$
3. Allocate resources temporarily and mark process as finished
4. Release resources after execution
5. Repeat until all processes are completed

If all processes complete → Safe State

Otherwise → Unsafe State

Code :

```
#include <iostream>
using namespace std;
int main() {
    int n = 5; // number of processes
    int m = 3; // number of resources
    int alloc[5][3] = { {0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2} };
    int max[5][3] = { {7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3} };
    int avail[3] = {3, 3, 2};
    int need[5][3];
    // Calculate Need matrix
    for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
    bool finish[5] = {false};
    int safeSeq[5];
    int count = 0;
    while (count < n) {
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                int j;
                for (j = 0; j < m; j++) {
                    if (need[i][j] > avail[j])
                        break;
                }
                // If all needs satisfied
                if (j == m) {
                    for (int k = 0; k < m; k++) {
                        avail[k] += alloc[i][k];
                    }
                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }
        if (!found) {
            cout << "System is NOT in safe state\n";
            return 0;
        }
    }
    cout << "System is in SAFE state\nSafe sequence: ";
    for (int i = 0; i < n; i++) {
        cout << "P" << safeSeq[i] << " ";
    }
    cout << endl;

    return 0;
}

```

Output :

```
System is in SAFE state  
Safe sequence: P1 P3 P4 P0 P2
```

Conclusion :

- The Banker's Algorithm ensures deadlock avoidance by checking system safety before allocation.
- It helps:
 - Prevent unsafe resource allocation
 - Maintain system stability
- The algorithm is widely used in resource management systems and OS scheduling.